

This homework is due by **11:59pm on March 20** via the Autograder page. Start early!

Instructions. Solutions must be *typed* in $\text{\LaTeX}$ (a template for this homework is available on the course web page). Your work will be graded on *correctness*, *clarity*, and *concision*. You should only submit work that you believe to be correct; if you cannot solve a problem completely, you will get significantly more partial credit if you clearly identify the gap(s) in your solution. It is good practice to start any long solution with an informal (but accurate) “proof summary” that describes the main idea.

You may consult external sources. However, you must **write your own solutions** and **list your sources** for each problem. If you turn in a proof you cannot explain orally, you will fail the assignment.

1. (*Discrete logarithms and Pedersen commitments.*)

Recall the discrete logarithm assumption we discussed in class. See pseudocode on the left-hand side below. Let $\mathbb{G}$ be a group of order p a prime and G be a generator of $\mathbb{G}$. (Remember that $\mathbb{G}$ having prime order means every element has order p .) We will denote the identity of $\mathbb{G}$ as 0.

We can generalize the task of finding the discrete logarithm of a single group element to the task of finding a discrete logarithm *relation* between many elements. To define this, first let $\text{msm}(\vec{x}, \vec{G}) = \sum_{i=1}^n \vec{x}_i \vec{G}_i$ denote the multi-scalar multiplication between a vector $\vec{x}$ of n $\mathbb{Z}_p$ elements and an equal-length vector $\vec{G}$ of elements of $\mathbb{G}$. Then, a discrete logarithm relation between n group elements $\vec{G} = (G_1, \dots, G_n)$ is a nonzero vector $\vec{x} = (x_1, \dots, x_n)$ of $\mathbb{Z}_p$ elements such that $\text{msm}(\vec{x}, \vec{G}) = 0$. The pseudocode on the right below defines a game $\text{DLREL}^{\mathbb{G}, p, n}$ that tasks an adversary $\mathcal{A}$ with finding a discrete logarithm relation between n random group elements. We denote the vector of all zeros as $\vec{0}$.

$\text{DL}^{\mathbb{G}, G, p}(\mathcal{A})$: $x \leftarrow \mathbb{Z}_p$ $x' \leftarrow \mathcal{A}(xG, \mathbb{G}, G, p)$ Return $x = x'$	$\text{DLREL}^{\mathbb{G}, p, n}(\mathcal{A})$: $\vec{G} \leftarrow \mathbb{G}^n$ $\vec{x} \leftarrow \mathcal{A}(\vec{G}, \mathbb{G}, p)$ Return $\text{msm}(\vec{x}, \vec{G}) = 0 \wedge \vec{x} \neq \vec{0}$
--	--

- Use a reduction to show that finding a discrete logarithm relation is as hard as finding the discrete logarithm of a single element. More concretely, you should construct a reduction $\mathcal{B}$ that, given access to an adversary $\mathcal{A}$ that wins $\text{DLREL}^{\mathbb{G}, p, n}$ with probability ϵ , wins game $\text{DL}^{\mathbb{G}, G, p}$ with probability at least ϵ (perhaps minus a small additional term).
 - The vector Pedersen commitment VPC we discussed in class is depicted in Figure 1, along with (on the right) the game CommBind defining binding security for a commitment CS. Use a reduction to show that breaking binding for VPC is as hard as finding a discrete logarithm relation in $\mathbb{G}$. More concretely, you should construct a reduction $\mathcal{B}$ that, given access to an adversary $\mathcal{A}$ that wins CommBind against VPC with probability ϵ , wins game $\text{DLREL}^{\mathbb{G}, p, n}$ with probability at least ϵ (perhaps minus a small additional term).
2. (*Pedersen commitments to bits.*) The project 3 handout describes a sigma protocol for proving a multiplication constraint between Pedersen-committed values—concretely, given C_a, C_b, C_c commitments to a, b, c , the protocol lets the prover show that $ab = c$ (without revealing a, b, c). Use this protocol to build a sigma protocol for proving that the value committed in a Pedersen commitment is either zero or one (without revealing which). Prove your protocol is 2-special-sound and special HVZK.
3. (*Compiling PIOPs to arguments.*) In this exercise you will prove the following theorem about the “oracle-to-PC” compiler we discussed in class.

$\text{VPC.Commit}(\text{pp}, \vec{x}):$ $\vec{G}, H, p \leftarrow \text{pp}$ $r \leftarrow \mathbb{Z}_p$ $C \leftarrow \text{msm}(\vec{x}, \vec{G}) + rH$ $\text{Return } C, r$	$\text{VPC.Open}(\text{pp}, C, \vec{x}, r):$ $\vec{G}, H, p \leftarrow \text{pp}$ $\text{Return } C = (\text{msm}(\vec{x}, \vec{G}) + rH)$	$\text{CommBind}^{\text{CS}}(\mathcal{A}):$ $\text{pp} \leftarrow \text{CS.Setup}$ $(x_0, r_0), (x_1, r_1), C^* \leftarrow \mathcal{A}(\text{pp})$ $\text{Return CS.Open}(\text{pp}, C^*, x_0, r_0)$ $\quad \wedge \text{CS.Open}(\text{pp}, C^*, x_1, r_1)$ $\quad \wedge (x_0, r_0) \neq (x_1, r_1)$
--	--	---

Figure 1: The vector Pedersen commitment VPC and the game CommBind defining binding security for a commitment CS.

Theorem. Let $\text{piop} = \langle P_{\text{piop}}, V_{\text{piop}} \rangle$ be a polynomial interactive oracle proof for $\mathcal{L}$ with degree and variable bounds $(d_1, v_1), \dots, (d_k, v_k)$. Let $\text{pc} = (\text{Setup}, \text{Commit}, \text{Open})$ be a polynomial commitment scheme. Let $\text{arg}[\text{piop}, \text{pc}]$ be the argument obtained from the “oracle-to-PC” compiler. If pc.Open is a proof of knowledge and piop is sound, then $\text{arg}[\text{piop}, \text{pc}]$ is a sound interactive argument for $\mathcal{L}$.

You will prove this using a hybrid argument: first, you will move from the standard soundness game SND for $\text{arg}[\text{piop}, \text{pc}]$ to a game G_1 where the challenger extracts a witness from the malicious prover after every PC opening. Then, you will show a malicious prover in this hybrid game G_1 can be turned into a malicious prover for the PIOP soundness game.

- Write the pseudocode for game G_1 . Prove that for any prover P^* , the difference in its win probability between SND and G_1 is at most the knowledge error of pc .
- Complete the proof by showing a reduction from game G_1 to the PIOP soundness game. More concretely, you should construct a reduction $\mathcal{B}$ that, given access to a prover P^* that wins game G_1 with probability ϵ , wins the PIOP soundness game with probability at least ϵ (perhaps minus a small additional term).
- (Extra credit.)

Your proof relies quite fundamentally on the knowledge soundness of pc ’s opening proof. For extra credit, either prove or disprove the following stronger claim that relaxes the assumed security of pc ’s opening proof:

Theorem. Let $\text{piop} = \langle P_{\text{piop}}, V_{\text{piop}} \rangle$ be a polynomial interactive oracle proof for $\mathcal{L}$ with degree and variable bounds $(d_1, v_1), \dots, (d_k, v_k)$. Let $\text{pc} = (\text{Setup}, \text{Commit}, \text{Open})$ be a polynomial commitment scheme. Let $\text{arg}[\text{piop}, \text{pc}]$ be the argument obtained from the “oracle-to-PC” compiler. If pc.Open **and** piop **are sound**, then $\text{arg}[\text{piop}, \text{pc}]$ is a sound interactive argument for $\mathcal{L}$.

4. (Fiat-Shamir soundness for sequential composition.)

This question illustrates the need for stronger soundness notions when compiling IPs to non-interactive arguments using the Fiat-Shamir transform. Let $\text{IP} = \langle \text{Setup}, P, V \rangle$ be a public-coin interactive proof for language $\mathcal{L}$ with soundness error $1/3$. Let IP_{seq}^n be the interactive proof for $\mathcal{L}$ obtained by n -fold sequential composition of IP . (In slightly more detail, IP_{seq}^n runs IP n times in sequence; its verifier returns 1 only if all n runs of IP return 1.) Note that IP_{seq}^n is public coin because IP is.

- Show that IP_{seq}^n has soundness error $1/3^n$.
- Show that the non-interactive argument $\text{FS}^{\text{RO}}[\text{IP}_{\text{seq}}^n]$ obtained by applying the Fiat-Shamir transform is *not* sound, by giving a very efficient attack against it. You need not give full pseudocode for your attack, but you must state and prove a precise claim about the attack’s success probability.

5. (*Parallel composition of interactive arguments of knowledge.*)

For any two interactive protocols $IP_1 = \langle \text{Setup}_1, P_1, V_1 \rangle$ and $IP_2 = \langle \text{Setup}_2, P_2, V_2 \rangle$ the parallel composition is the interactive protocol where the i th prover message is the concatenation of the i th prover messages of IP_1 and IP_2 , and likewise for the verifier messages. The verifier's output is the AND of the verifier outputs of IP_1 and IP_2 .

Let AoK_1 be an interactive argument of knowledge for relation $\mathcal{R}_1$ and AoK_2 be an interactive argument of knowledge for relation $\mathcal{R}_2$. Show that the parallel composition of AoK_1 and AoK_2 is an interactive argument of knowledge for relation $\mathcal{R}_1 \times \mathcal{R}_2$. (For concreteness, the relation $\mathcal{R}_1 \times \mathcal{R}_2$ is the set of pairs of pairs $(x_1, x_2), (w_1, w_2)$ such that $(x_1, w_1) \in \mathcal{R}_1$ and $(x_2, w_2) \in \mathcal{R}_2$.)